



FIXFLIP

شرح مسار الطلب والاستجابة في مشروع ASP.NET Web API

Layered Architecture - Controller -> Service -> Data -> Response

هدف الدليل: شرح رحلة الطلب من العميل إلى قاعدة البيانات، ثم توضيح كيف تعود النتيجة عبر طبقات المشروع حتى تصل إلى المستخدم.

ملخص المسار الكامل

REQUEST PATH Client / Postman -> HTTP Request -> Controller Layer -> Interface Layer -> Service Layer -> ApplicationDbContext -> Entity Framework Core -> SQL Database RESPONSE PATH SQL Database -> Entity Framework Core -> ApplicationDbContext -> Service Layer -> DTO Layer -> Controller Layer -> HTTP Response -> Client Receives Data

إعداد لعرض ومناقشة مشروع FixFlip

نظرة عامة على المعمارية

GENERAL BACKEND ARCHITECTURE

يعتمد المشروع على **Layered Architecture**، أي أن كل طبقة لها مسؤولية واضحة. الطلب لا ينتقل مباشرة من Controller إلى قاعدة البيانات، بل يمر عبر العقود والخدمات وطبقة الوصول إلى البيانات.

الفكرة الأساسية: فصل المسؤوليات يجعل الكود أسهل في الفهم والاختبار والصيانة، ويمنع تداخل منطق الـ API مع منطق الأعمال أو قاعدة البيانات.

القسم الأول: مسار الطلب

REQUEST PATH - GREEN

يمثل الخط الأخضر الطلب أثناء دخوله إلى النظام. يبدأ من المستخدم أو Postman وينتهي بتنفيذ العملية داخل SQL Database.

Client / Postman 01 مصدر الطلب

هذه هي الجهة التي تبدأ العملية وترسل طلبًا إلى الـ API. قد تكون واجهة المستخدم، تطبيقًا آخر، أو أداة Postman أثناء الاختبار.

أمثلة: طلب قائمة السيارات، أو إضافة علامة تجارية جديدة.

```
GET /api/car
POST /api/brand
```

HTTP Request 02 بنية الطلب

يتحول طلب المستخدم إلى HTTP Request كامل يحمل المعلومات التي يحتاجها السيرفر لفهم العملية وتنفيذها.

العنصر	المحتوى
Method	GET / POST / PUT / DELETE
URL	/api/car
Headers	مثل Token أو Content-Type
Body	البيانات المرسلّة في POST أو PUT

مثال على Body بصيغة JSON

```
{
  "name": "Toyota"
}
```

مسار الطلب: طبقات التطبيق

CONTROLLER - INTERFACE - SERVICE

03 Controller Layer طبقة استقبال الطلب

أول مكان داخل الكود يستقبل HTTP Request. من أمثلتها **CarController** و **BrandController** و **AuthController**.

مسؤوليته: استقبال الطلب، قراءة البيانات، استدعاء Service Layer، ثم إعادة HTTP Response.

قاعدة مهمة: Controller لا يتعامل مع Database مباشرة. دوره تنسيق الاتصال بين HTTP والطبقات الداخلية، وليس تنفيذ منطق الأعمال.

04 Interface Layer العقود

تمثل هذه الطبقة العقد الذي يحدد العمليات المتاحة بدون كتابة طريقة التنفيذ.

أمثلة: ICarService، IBrandService، IModelService.

عمليات شائعة: GetAll، GetById، Create، Update، Delete.

المعنى: الـ Interface يصف ما الذي يمكن تنفيذه، بينما Service Layer نكتب كيف يتم التنفيذ فعليًا.

05 Service Layer منطق الأعمال

هنا يوجد التنفيذ الحقيقي للعملية. تحتوي هذه الطبقة على Business Logic وقواعد التحقق قبل الوصول إلى البيانات.

مسؤولياتها: Validation، استدعاء ApplicationDbContext، تطبيق قواعد النظام، وتحضير البيانات.

أمثلة تحقق: التأكد أن TrimId صحيح، وأن UserId موجود، وأن العملية مسموحة.

مسار الطلب: الوصول إلى البيانات

DBCONTEXT - ENTITY FRAMEWORK CORE - SQL

AppDbContext 06 طبقة الوصول إلى البيانات

يمثل حلقة الوصل بين كود المشروع وقاعدة البيانات. يحتوي على DbSet لكل Entity أو جدول يحتاجه النظام.

مثال

```
_context.Cars
```

المعنى: الوصول إلى مجموعة Cars التي تمثل جدول السيارات داخل قاعدة البيانات.

ORM Entity Framework Core 07

هو Object-Relational Mapper يحول أوامر #C و LINQ إلى أوامر SQL يمكن لقاعدة البيانات تنفيذها.

الكود في #C

```
_context.Cars.ToListAsync();
```

الاستعلام التقريبي الناتج

```
SELECT * FROM Cars;
```

SQL Database 08 التخزين الفعلي

هنا تُخزن بيانات النظام فعليًا، وتُنفذ عمليات القراءة والإضافة والتعديل والحذف.

أمثلة الجداول: Users، Brands، Models، Trims، Cars، Sales، Tasks، CarImages.

العمليات: SELECT للقراءة، INSERT للإضافة، UPDATE للتعديل، DELETE للحذف.

نهاية المسار الأخضر: عند هذه النقطة تكون العملية المطلوبة قد وصلت إلى قاعدة البيانات وتم تنفيذ الاستعلام المناسب.

القسم الثاني: مسار الاستجابة

RESPONSE PATH - PURPLE

يمثل الخط البنفسجي البيانات أثناء عودتها من SQL Database إلى المستخدم. تتحول النتائج إلى Objects، ثم تُجهز وتُحوّل إلى DTO قبل إرسالها كـ HTTP Response.

Entity Framework Core 09

تحويل النتائج

بعد أن تعيد SQL Database النتيجة، يحول EF Core الصفوف القادمة من SQL إلى Objects يمكن التعامل معها داخل #C.

Rows from SQL



Objects in C#

AppDbContext 10

إرجاع البيانات

يستقبل النتائج التي جهزها EF Core ويعيدها إلى Service Layer بصيغة Entities أو Collections.

Service Layer 11

تجهيز النتيجة

تطبق الخدمة منطق النتيجة قبل إرجاعها. قد تحدد نجاح العملية أو فشلها وتجهز البيانات للمرحلة التالية.

مثال: إذا لم توجد السيارة تُرجع نتيجة فشل؛ وإذا وجدت تتابع تجهيز البيانات.

اتجاه العودة: في هذه المرحلة لا يتم تنفيذ طلب جديد؛ بل يتم تمرير نتيجة العملية إلى الأعلى عبر طبقات المشروع.

مسار الاستجابة: تجهيز وإرسال البيانات

DTO - CONTROLLER - HTTP RESPONSE - CLIENT

12 DTO Layer

تحويل شكل البيانات

اختصار لـ Data Transfer Object. وظيفته تحديد شكل البيانات التي ستخرج من الـ API بدل إعادة الـ Entity أو Model كاملاً.

Car Entity
↓
CarDto

لماذا نستخدم DTO؟ لتنظيم الاستجابة، وحماية الحقول الداخلية، وتقليل حجم البيانات، ومنع إرجاع العلاقات الكاملة أو المعلومات غير المطلوبة.

13 Controller Layer

اختيار نوع الرد

تعود النتيجة إلى Controller، الذي يحولها إلى HTTP Result مناسب حسب حالة العملية.

Ok(data)
NotFound()
BadRequest()
Unauthorized()

14 HTTP Response

الرد النهائي

يتكون الرد النهائي من HTTP Status Code، وقد يحتوي على JSON إذا كانت هناك بيانات يجب إعادتها.

المعنى	Status Code
تمت العملية بنجاح	200 OK
المورد المطلوب غير موجود	404 Not Found
الطلب أو البيانات غير صحيحة	400 Bad Request
المستخدم غير مصرح له	401 Unauthorized

15 Client Receives Data

نهاية الرحلة

يستلم المستخدم أو Postman الاستجابة، ويعرض البيانات أو يتعامل مع حالة الخطأ حسب Status Code.

```
{
  "brandName": "Toyota",
  "modelName": "Camry",
  "year": 2020
}
```

ملخص جاهز للمناقشة

DEFENSE TALKING POINTS

يمكن شرح الرسم أمام لجنة المناقشة بالنص التالي:

يبدأ الطلب من Client أو Postman كـ HTTP Request، ويستقبله Controller Layer. بعد ذلك يستدعي Controller العقد الموجود في Interface Layer، ثم تنفذ Service Layer منطق الأعمال والتحقق. تستخدم الخدمة ApplicationDbContext، ويحول Entity Framework Core أو امر LINQ إلى SQL لتنفيذها داخل قاعدة البيانات.

عند عودة النتيجة، يحول EF Core الصفوف إلى Objects، ثم تمر عبر ApplicationDbContext و Service Layer. بعدها ترتب DTO Layer شكل البيانات وتحمي الحقول الداخلية، ويختار Controller نوع الاستجابة المناسب قبل إرسال HTTP Response إلى العميل.

أهم النقاط التي يجب التأكيد عليها

فصل المسؤوليات

كل طبقة تنفذ مهمة محددة، وهذا يحسن الصيانة والاختبار.

عدم تجاوز الطبقات

Controller لا يصل إلى Database مباشرة؛ بل يمر عبر Service و ApplicationDbContext.

دور Interface

تحدد العقود والعمليات بدون ربط Controller بتنفيذ Service محدد.

دور EF Core

يعمل كوسيط ORM بين Objects في C# والجداول والاستعلامات في SQL.

دور DTO

يتحكم في شكل البيانات الخارجة ويحمي تفاصيل Entity الداخلية.

الاستجابة القياسية

Controller يعيد Status Code مناسبًا مع JSON عند الحاجة.

جملة ختامية مقترحة

هذا التصميم يطبق Layered Architecture ويضمن أن مسار الطلب والاستجابة واضح، منظم، وقابل للتوسع عند إضافة Features جديدة إلى مشروع FixFlip.